

DP入门-板子

[P1002 \[NOIP 2002 普及组\] 过河卒 - 洛谷](#)

Level1: 状态转移方程: $f(i, j) = f(i - 1, j) + f(i, j - 1)$

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;
#define all(a) (a).begin(), (a).end()
const int fx[] = {0, -2, -1, 1, 2, 2, 1, -1, -2};
const int fy[] = {0, 1, 2, 2, 1, -1, -2, -2, -1};
ll f[40][40]; //二维数组存储方法数
int s[40][40]; //二维数组存储该位置有没有马

void work(){
    int x1, y1, xm, ym;
    cin >> x1 >> y1 >> xm >> ym;
    x1 += 2, y1 += 2, xm += 2, ym += 2; // 因为计算马的位置会“-2” 所以坐标+2防止越界
    f[1][2] = 1; // 初始化
    for(int i=0; i<=8; ++i) ++s[xm + fx[i]][ym + fy[i]]; // 标记马的位置

    for(int i=2; i<=x1; ++i){
        for(int j=2; j<=y1; ++j){
            if(s[i][j]) continue; // 如果被马拦住就直接跳过
            f[i][j] = f[i-1][j] + f[i][j-1];
        }
    }
    cout << f[x1][y1];
} signed main(){
    ios::sync_with_stdio(0);
    cin.tie(0);
    cout.tie(0);
    int T=1;
    // cin >> T;
    while(T--) work();
}

/*
6 6 3 3
6
*/
```

Level2: 考虑滚动数组优化

由上述状态转移方程可知：对于一次转移，我们只需要用到横坐标是 i 和横坐标是 $i-1$ 这两行的答案，其他位置的答案已经是没有用处的了

新的转移方程: $f(i \& 1, j) = f((i-1) \& 1, j) + f(i \& 1, j-1)$

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
#define all(a) (a).begin(), (a).end()
const int fx[] = {0, -2, -1, 1, 2, 2, 1, -1, -2}; const int fy[] = {0, 1, 2, 2, 1, -1, -2, -2, -1};
ll f[2][40];
int s[40][40];

void work(){
    int x1, y1, xm, ym;
    cin >> x1 >> y1 >> xm >> ym;
    x1 += 2, y1 += 2, xm += 2, ym += 2;
    f[1][2] = 1; // 第一维大小为 2 就好
    for(int i=0; i<=8; ++i) ++s[xm + fx[i]][ym + fy[i]];

    for(int i=2; i<=x1; ++i){
        for(int j=2; j<=y1; ++j){
            if(s[i][j]){
                f[i & 1][j] = 0; // 被马拦住了记住清零
                continue;
            }
            f[i & 1][j] = f[(i-1) & 1][j] + f[i & 1][j-1]; // 新的状态转移方程
        }
    }
    cout << f[x1 & 1][y1]; // 输出的时候第一维也要按位与一下
} signed main(){
    ios::sync_with_stdio(0);
    cin.tie(0);
    cout.tie(0);
    int T=1;
    // cin >> T;
    while(T--) work();
}

```

Level3: 考虑优化成一维数组

观察我们能发现, 这个 $f(i-1, j)$ 与当前位置的 $f(i, j)$ 的第二维一样, 都是 j , 而第一维只是差了 1

最终状态转移方程: $f(j) = f(j) + f(j-1)$

解释: $f(j)+f(j-1)$ 里面, $f(j-1)$ 就是前面方程里的 $f(i, j-1)$ 。

至于 $f(j)$, 因为还没有被更新过, 所以答案仍然保存的是上次求出的答案, 即 $f(i-1, j)$ 。

这样, 就把二维数组成功变成了一维数组。

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
#define all(a) (a).begin(), (a).end()
const int fx[] = {0, -2, -1, 1, 2, 2, 1, -1, -2}; const int fy[] = {0, 1, 2, 2, 1, -1, -2,
-2, -1};
ll f[40]; //一维数组啦啦啦
int s[40][40];

void work(){
    int x1, y1, xm, ym;
    cin >> x1 >> y1 >> xm >> ym;
    x1 += 2, y1 += 2, xm += 2, ym += 2;
    f[1] = 1; //初始化
    for(int i=0; i<=8; ++i) ++s[xm + fx[i]][ym + fy[i]];
    for(int i=2; i<=x1; ++i){
        for(int j=2; j<=y1; ++j){
            if(s[i][j]){
                f[j] = 0; // 被马拦住了记住清零
                continue;
            }
            f[j] += f[j-1]; // 全新的状态转移方程
        }
    }
    cout << f[y1];
} signed main(){
    ios::sync_with_stdio(0);
    cin.tie(0);
    cout.tie(0);
    int T=1;
    //    cin >> T;
    while(T--) work();
}

```